

MÓDULO 6: Lifetimes (Tiempos de Vida)

Este módulo cierra el círculo abierto en el Módulo 2. Ya sabes que el borrow checker impide dangling references. Los **lifetimes** son la herramienta que usa el compilador —y que tú a veces debes hacer explícita— para verificar eso cuando la relación entre referencias no es obvia.

🔑 **Idea clave que debes internalizar primero:** los lifetimes **no cambian cuánto vive nada**. No extienden ni acortan la vida de ningún valor. Son **anotaciones que describen** una relación que ya existe en tu código, para que el compilador pueda verificarla. Es información, no control.

6.1 El Problema que Resuelven

```
fn mas_largo(x: &str, y: &str) -> &str { // ❌
    if x.len() > y.len() { x } else { y }
}
error[E0106]: missing lifetime specifier
--> src/main.rs:1:37
   |
1  | fn mas_largo(x: &str, y: &str) -> &str {
   |                                     ^ expected named lifetime parameter
= help: this function's return type contains a borrowed value, but the
signature does not say whether it is borrowed from `x` or `y`
```

Por qué el compilador no puede adivinar solo: la función devuelve `&str`, una referencia. Pero, ¿a cuál de las dos entradas apunta esa referencia? Dependiendo del valor de `x.len() > y.len()`, podría ser cualquiera de las dos, y eso solo se sabe en runtime. El compilador necesita saber, **en compile-time**, cuál es la relación de vida entre las entradas y la salida, para poder verificar que quien reciba el resultado no lo use después de que la fuente original haya muerto.

6.2 Sintaxis de Lifetimes

```
fn mas_largo<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("string largo");
    let resultado;
    {
        let s2 = String::from("corto");
        resultado = mas_largo(s1.as_str(), s2.as_str());
        println!("El más largo es: {}", resultado); // ✅ s2 aún
vive aquí
    }
    // println!("{}", resultado); // ❌ si lo movieras aquí,
```

```
fallaría (s2 ya murió)
}
```

Cómo leer `<'a>(x: &'a str, y: &'a str) -> &'a str`: *"existe algún lifetime 'a tal que x, y, y el valor de retorno viven al menos tanto como 'a"*. No dice que x e y tengan que vivir exactamente lo mismo — dice que el retorno es válido solo mientras **ambos** sigan siendo válidos (el compilador toma el lifetime **más corto** de los dos como límite seguro).

Diagrama mental

```

      |<----- 'a ----->|
s1:  |=====| (vive mucho)
s2:  |=====| (vive menos)
resultado: solo es válido dentro de la intersección —
          (el compilador usa el lifetime MÁS CORTO como límite)

```

Error típico del compilador #1: Usar el resultado fuera del lifetime válido

```
fn main() {
    let s1 = String::from("largo");
    let resultado;
    {
        let s2 = String::from("corto");
        resultado = mas_largo(s1.as_str(), s2.as_str());
    } // s2 muere aquí
    println!("{}", resultado); // ❌
}
```

```
error[E0597]: `s2` does not live long enough
--> src/main.rs:6:38
   |
5 |         let s2 = String::from("corto");
   |         -- binding `s2` declared here
6 |         resultado = mas_largo(s1.as_str(), s2.as_str());
   |                                ^^^^^^^^^^^^^^^^^ borrowed value
does not live long enough
7 |     }
   |     - `s2` dropped here while still borrowed
8 |     println!("{}", resultado);
   |                       ----- borrow later used here

```

Esto es exactamente el mismo tipo de bug del dangling reference del Módulo 2, pero detectado a través de la cadena de lifetimes en vez de un caso trivial de una sola función.

6.3 Reglas de Elisión de Lifetimes

Si has escrito funciones con `&str/&T` sin nunca poner `'a`, es porque el compilador aplica **reglas de elisión** automáticamente en casos comunes:

```
// Estas dos firmas son EQUIVALENTES – el compilador infiere la
segunda:
fn primera_palabra(s: &str) -> &str { /* ... */ }
fn primera_palabra_explicita<'a>(s: &'a str) -> &'a str { /* ... */
}
```

Las 3 reglas de elisión (si todas se satisfacen, no necesitas anotar nada):

1. Cada parámetro por referencia recibe su propio lifetime implícito.
2. Si hay **exactamente un** parámetro de entrada por referencia, ese lifetime se asigna a todos los retornos por referencia.
3. Si hay **&self o &mut self** (es un método), el lifetime de self se asigna a todos los retornos por referencia.

```
struct Extractor<'a> {
    texto: &'a str,
}

impl<'a> Extractor<'a> {
    // Regla 3 aplica: no necesitas anotar el retorno, se infiere de
    &self
    fn primera_linea(&self) -> &str {
        self.texto.lines().next().unwrap_or("")
    }
}
```

Solo necesitas anotar `'a` manualmente cuando el compilador **no puede aplicar ninguna** de estas 3 reglas — típicamente cuando hay 2+ parámetros por referencia y el retorno depende de la relación entre ellos, como en `mas_largo`.

6.3.b Modelado Visual de Líneas de Vida Cruzadas en Retornos

Cuando una función toma múltiples referencias de entrada y retorna una referencia, anotar los lifetimes incorrectamente puede encadenar dependencias innecesarias que bloquean variables en el llamador.

Diagrama Mental: Lifetimes Independientes vs Acoplados

CASO 1: ACOPLAMIENTO FORZADO (Antipatrón innecesario)

```
fn buscar<'a>(buffer: &'a str, patron: &'a str) -> &'a str
    buffer: |=====| ('a)
    patron:  |=====| ('a)
    retorno: solo es válido mientras `patron` viva, ¡aunque el retorno apunte a `buffer`!
```

CASO 2: DESACOPLAMIENTO IDIOMÁTICO

```
fn buscar<'a, 'b>(buffer: &'a str, _patron: &'b str) -> &'a str
```

```

buffer: |=====| ('a)
patron:  |=====|      ('b - muere sin afectar al retorno)
retorno: |=====| (asociado estrictamente al scope de buffer)

```

Código Compilable

```

// El retorno proviene exclusivamente del haystack (buffer), la aguja (patron) es descartable
fn extraer_prefijo<'a, 'b>(buffer: &'a str, delimitador: &'b str) -> &'a str {
  if let Some(pos) = buffer.find(delimitador) {
    &buffer[..pos]
  } else {
    buffer
  }
}

fn main() {
  let buffer_largo = String::from("clave_primaria:valor_secundario");
  let prefijo: &str;

  {
    // Delimitador vive en un scope reducido
    let delim_temporal = String::from(":");
    prefijo = extraer_prefijo(&buffer_largo, &delim_temporal);
  } // <-- delim_temporal muere aquí

  // ✅ Totalmente legal: `prefijo` solo está atado al lifetime del `buffer_largo`
  println!("Prefijo extraído con éxito: {}", prefijo);
}

```

6.4 Structs con Referencias

Cuando un struct **guarda una referencia** en vez de ser dueño del dato, necesita anotar el lifetime:

```

struct Extracto<'a> {
  parte: &'a str, // "Extracto no puede vivir más que el str al
que apunta"
}

impl<'a> Extracto<'a> {
  fn nuevo(texto: &'a str) -> Extracto<'a> {
    let primera = texto.split('.').next().unwrap_or(texto);
    Extracto { parte: primera }
  }
}

fn main() {
  let novela = String::from("Era una noche oscura. El viento
soplaba fuerte.");
  let extracto = Extracto::nuevo(&novela);
  println!("Extracto: {}", extracto.parte);
}

```

```
} // extracto y novela mueren juntos aquí, sin problema
```

Diagrama mental

```
novela: String (owner) —————  
extracto: Extracto<'a> { parte: &'a str } ————— apunta a  
datos DENTRO de novela
```

REGLA: extracto NO PUEDE sobrevivir a novela.
El compilador rechaza cualquier código donde extracto se use después de que novela salga de scope.

Error típico del compilador #2: Struct sobreviviendo a su referencia

```
struct Extracto<'a> {  
    parte: &'a str,  
}  
  
fn crear_extracto() -> Extracto<'static> { // pidiendo 'static a  
propósito para forzar el error  
    let texto = String::from("temporal");  
    Extracto { parte: &texto } // X  
} // texto muere aquí, pero prometimos devolver algo 'static  
  
error[E0515]: cannot return value referencing local variable `texto`  
--> src/main.rs:7:5  
7 |         Extracto { parte: &texto }  
  |         -----^^^^^^  
  |         |  
  |         `texto` is borrowed here  
  |         returns a value referencing data owned by the current  
function
```

Solución real: si necesitas devolver algo que sobreviva a la función, la función debe **crear y devolver el dato con ownership** (String), no una referencia a algo local. Los structs con lifetimes solo tienen sentido cuando el dato referenciado vive **fuera y más tiempo** que la estructura que lo referencia — típicamente en parsers que devuelven "vistas" (slices) sobre un buffer que el llamador posee.

6.5 El Lifetime 'static

```
fn main() {  
    let s: &'static str = "esto vive durante TODO el programa"; //
```

```
literales de texto son 'static
println!("{}", s);
}
```

'static significa que la referencia es válida durante **toda la duración del programa**. Los literales de texto ("...") son 'static porque están embebidos directamente en el binario compilado.

⚠ **Advertencia común para principiantes:** cuando el compilador sugiere 'static como solución a un error de lifetime, casi siempre es una señal de que estás modelando algo mal (ej: intentando devolver una referencia a un dato local), no la solución correcta. Úsalo con intención, no como "parche mágico para que compile".

6.6 Combinando Lifetimes con Trait Bounds

```
use std::fmt::Display;

fn anunciar_y_mas_largo<'a, T>(x: &'a str, y: &'a str, anuncio: T)
-> &'a str
where
    T: Display,
{
    println!("{}", "¡Atención! {}", anuncio);
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let s1 = String::from("largo texto");
    let s2 = String::from("corto");
    let resultado = anunciar_y_mas_largo(&s1, &s2, "Comparando
strings");
    println!("{}", "Resultado: {}", resultado);
}
```

6.6.b Subtipado y Bounds de Lifetimes ('a: 'b)

La sintaxis 'a: 'b se lee como: **"el lifetime 'a sobrevive al menos tanto como 'b"** (o 'a es un *subtipo* de 'b).

Es indispensable cuando una estructura o función contiene dos lifetimes y una referencia interna debe apuntar o contener a la otra sin riesgo de invalidación de memoria.

Diagrama Mental: Relación de Subtipado

Línea de tiempo de memoria:

```
'a: |=====| (Sobrevive más)
'b: |=====| (Vive menos)
      'a: 'b garantiza que 'a cubre toda la existencia de 'b.
```

Código Compilable

```
struct Contexto<'a> {
    texto: &'a str,
}
```

```
// Struct con dos lifetimes dependientes: 'a debe sobrevivir a 'b
('a: 'b)
struct Procesador<'a, 'b: 'a> {
    contexto: &'a Contexto<'b>,
}

impl<'a, 'b: 'a> Procesador<'a, 'b> {
    fn obtener_fragmento(&self) -> &'b str {
        // Retornar con lifetime 'b es seguro porque 'b sobrevive o
        // iguala a 'a
        self.contexto.texto
    }
}

fn main() {
    let texto_base = String::from("datos globales de telemetría");
    let ctx = Contexto { texto: &texto_base };

    let fragmento;
    {
        let proc = Procesador { contexto: &ctx };
        fragmento = proc.obtener_fragmento();
    } // `proc` muere aquí

    println!("Fragmento retenido: {}", fragmento); // Sigue siendo
    válido}

```

6.6.c Higher-Ranked Trait Bounds (HRTBs) con for<'a>

En la mayoría de las funciones genéricas, los lifetimes son elegidos por el llamador. Sin embargo, cuando trabajas con closures o parsers de bajo nivel, a menudo necesitas expresar que un closure o trait bound debe ser válido para **cualquier lifetime posible** que se genere internamente en tiempo de ejecución.

La sintaxis for<'a> formaliza este contrato de rango superior (*Higher-Ranked Trait Bound*):

for<'a> F: Fn(&'a str) -> &'a str

"Para cualquier lifetime 'a que exista en el universo, F puede aceptar un &'a str y retornar un &'a str".

Código Compilable

```
// Acepta un closure que pueda procesar referencias con CUALQUIER
lifetime dinámico
fn aplicar_transformacion<F>(linea: &str, parser: F)
where
    F: for<'a> Fn(&'a str) -> &'a str, // HRTB explícito
{
    let resultado = parser(linea);
    println!("Línea procesada: {}", resultado);
}

```

```

}

fn main() {
    let log = "2026-08-29 INFO: Inicialización correcta";

    // El closure procesa el slice temporal sin importar el lifetime
    exacto
    aplicar_transformacion(log, |s| {
        s.split_whitespace().last().unwrap_or(s)
    });
}

```

6.7 Resolviendo buscar_por_autor del Módulo 4

Recordemos la función pendiente:

```

struct Libro {
    titulo: String,
    autor: String,
}

struct Biblioteca {
    libros: Vec<Libro>,
}

impl Biblioteca {
    fn buscar_por_autor<'a>(&'a self, autor: &str) -> Vec<&'a Libro>
    {
        self.libros.iter().filter(|libro| libro.autor ==
autor).collect()
    }
}

fn main() {
    let biblio = Biblioteca {
        libros: vec![
            Libro { titulo: String::from("Dune"), autor:
String::from("Herbert") },
            Libro { titulo: String::from("1984"), autor:
String::from("Orwell") },
        ],
    };
    let resultados = biblio.buscar_por_autor("Herbert");
    for libro in &resultados {
        println!("{}", libro.titulo);
    }
} // resultados NO puede sobrevivir a biblio, y el compilador lo
garantiza

```

Todo encaja ahora con claridad: *"El lifetime 'a establece que cada referencia &Libro devuelta permanecerá válida únicamente mientras dure el préstamo de &self"*. Por este motivo, si intentaras almacenar los resultados y destruir la instancia con `drop(biblio)`, el compilador bloquearía la operación de inmediato. Se trata de la misma salvaguarda contra

referencias colgantes (dangling references) introducida en el Módulo 2, ahora articulada para gestionar la relación entre un struct y sus datos internos.

6.8 Tabla Resumen: Cuándo Necesitas Anotar Lifetimes Explícitamente

Situación	¿Necesitas anotar 'a?
Función con 1 solo parámetro por referencia, retorna referencia	No (regla de elisión 2)
Método con &self, retorna referencia derivada de self	No (regla de elisión 3)
Función con 2+ parámetros por referencia, retorno depende de ambos	Sí
Struct que almacena una referencia como campo	Sí, siempre
Todos los datos son owned (String, Vec<T>, no &str/&[T])	No, nunca (no hay referencias que rastrear)

💡 **Consejo práctico de diseño:** cuando estás empezando, si te encuentras peleando mucho con lifetimes en un struct, la solución pragmática suele ser **usar tipos owned** (String en vez de &str, Vec<T> en vez de &[T]) a costa de algún .clone() extra. Optimizas para lifetimes complejos solo cuando el profiling te dice que las copias son realmente un problema de rendimiento. Esto es un consejo real de ingenieros senior de Rust: "cuando dudes, clona; optimiza después".

🎯 Ejercicio Integrador — Módulo 6

Proyecto: Parser de Logs con Vistas Zero-Copy

Crea cargo new parser_logs. El objetivo es procesar un log de texto **sin copiar strings**, usando slices con lifetimes — un patrón muy real en parsers de alto rendimiento (y antesala directa de lo que harás con nom en el Módulo 12).

- Define:

```
struct EntradaLog<'a> {
    timestamp: &'a str,
    nivel: &'a str,
    mensaje: &'a str,
}
```
- Implementa fn parsear_linea<'a>(linea: &'a str) -> Option<EntradaLog<'a>> que reciba una línea con formato "2024-01-15 ERROR conexión rechazada" (separado por espacios, donde el mensaje es todo lo que sobra) y devuelva None si la línea no tiene al menos 3 partes.
- Implementa fn filtrar_por_nivel<'a>(entradas: &'a [EntradaLog<'a>], nivel: &str) -> Vec<&'a EntradaLog<'a>>.
- En main:
 - Define un &str multilínea con 5-6 líneas de log simuladas (usa un string literal con \n).
 - Parsea cada línea con .lines() + parsear_linea, descartando los None con filter_map.
 - Filtra las entradas de nivel "ERROR" y muéstralas.
 - Verifica el zero-copy:** imprime con {:p} la dirección de memoria de algún &str del log original y de la parte extraída en EntradaLog, para comprobar que apuntan dentro del mismo buffer (no hay heap allocation nueva).

Reto de borrow checker: intenta hacer que la función main modifique (mueva o dropee) el string original ANTES de usar las EntradaLog extraídas, y observa el error de lifetime resultante. Pégamelo si quieres que lo diseccionemos línea por línea.

Este ejercicio es intencionalmente el más "denso" en lifetimes de todo el curso — si te cuesta, es completamente normal, es la parte del lenguaje con mayor curva de aprendizaje. Cuando lo tengas (o

si te atoras), compárteme tu código.

Ejercicio Integrador 2

Proyecto: Motor de Tokenización Zero-Copy con Subtipado y HRTBs

Crea cargo new motor_tokenizador e implementa:

1. **Parser Zero-Copy con Múltiples Lifetimes:**
 - Define struct Parser<'a, 'b: 'a> que almacene un buffer &'b str y un delimitador de regla &'a str.
 - Implementa fn siguiente_token(&mut self) -> Option<&'b str> asegurando que el token retornado tenga la vida útil del buffer, no del parser.
2. **Aplicación de HRTBs:**
 - Escribe una función de procesamiento fn procesar_stream<F>(stream: &[&str], tokenizador: F) donde F: for<'k> Fn(&'k str) -> Option<&'k str>.
3. **Validación en main:**
 - Procesa un array de strings estáticos y temporales, verificando que los tokens extraídos no clonen memoria (comprobando sus punteros de inicio con {:p}) y que no existan interferencias entre lifetimes desacoplados.